

Técnicas de Programação I – Professor Henrique Louro

Resumo para Estudos Prova 2 – 2025-2

I. Introdução e Ambiente (Node.js e TypeScript)

- Node.js: Plataforma de execução que permite rodar código JavaScript (JS) fora do navegador, usando o motor V8 do Google Chrome.
- Características: Utiliza arquitetura orientada a eventos e I/O não bloqueante para eficiência e escalabilidade (back-end, APIs RESTful).
- NPM (Node Package Manager): Gerenciador de pacotes para instalar, gerenciar e compartilhar bibliotecas.
- Partes do NPM: Website (documentação), Command Line Interface (CLI) (ferramenta de download/instalação/gerenciamento), e Registro (repositório).
- Comandos Comuns: `npm install <pacote>` (instala localmente); `npm i <pacote> -D` (dependência de desenvolvimento); `npm i <pacote> -g` (instala globalmente); `npx` (executa pacotes Node).
- TypeScript (TS): É um superset do JS, adicionando recursos de tipagem estática.
- Tipagem Estática vs. Dinâmica: No TS, as variáveis têm tipo definido na declaração e não podem ser alteradas (tipagem estática), diferentemente do JS (tipagem dinâmica). A vantagem é detectar erros em tempo de compilação/digitação.
- Compilação: O código TS precisa ser compilado para JS puro através do compilador `tsc` (TypeScript Compiler) antes da execução.
- `tsconfig.json`: Arquivo de configuração que define as opções do compilador, como a versão do ECMAScript (ES) de destino (target).
- Execução: Pode ser feita manualmente (`tsc` seguido por `node`) ou usando ferramentas como `ts-node` (que compila e executa diretamente, geralmente usada como dependência de desenvolvimento).

II. Variáveis, Tipos de Dados e Funções

- Declaração de Variável:
 - Inferência de Tipo: O TS infere o tipo com base no valor atribuído (ex: `let nome = "Ana"` infere `string`).
 - Anotação Explícita: O tipo é explicitamente anotado (ex: `let nome:string = "Ana"`).
 - Tipos Primitivos: `string`, `number` (inteiros ou reais), `boolean`, `null`, `undefined`, e `bigint` (para números grandes).
 - Tipo Especial `any`: Desabilita a verificação de tipo estática, permitindo qualquer valor, mas seu uso excessivo é desencorajado por comprometer a segurança e robustez do código.
 - Arrays: Podem ser declarados com anotação de tipo e colchetes (ex: `number[]`) ou usando `*generics*` (ex: `Array<number>`).

Técnicas de Programação I – Professor Henrique Louro

- União de Tipos: Combina dois ou mais tipos usando o operador pipe (`|`), permitindo que a variável aceite diferentes tipos de valores (ex: `let valor:number|string`).
- **Funções: Podem ser definidas de três formas:**
 1. Nomeada: Usa `function` seguido pelo nome.
 2. Anônima: Atribuída a uma variável (expressão de função).
 3. Arrow Function: Sintaxe curta, sempre anônima. Se tiver apenas `return` no corpo, chaves e `return` podem ser omitidos.
- Anotação de Tipos em Funções: Permite definir tipos para parâmetros e para o retorno. O retorno é colocado após o parêntese dos parâmetros.
- Parâmetros Condicionais: Definidos com `?` (ex: `b?:number`). O parâmetro terá valor `undefined` se não for passado na chamada.
- Métodos de Array (JS/TS):
 - `forEach`: Itera sobre os elementos, não possui retorno.
 - `map`: Invoca a função de `*call-back*` para cada elemento e retorna um novo array com o resultado, sem modificar o original.
 - `reduce`: Executa a função de `*call-back*` e retorna um único valor (ex: somatório), sem modificar o original.
- Módulos (`Export` e `Import`): Usados para modularização e reutilização de código.
- Tipos de Exportação: Por padrão (`export default`, apenas uma entidade por arquivo), Nomeada (`export function...` ou `export const...`), e Agrupada (`export { msg, resposta }`).

III. Classes e Encapsulamento

- Classes e Objetos: A classe é um template (modelo) que define um tipo de dado composto. O objeto (ou instância) é uma cópia da classe criada usando `new`.
- Membros: Variáveis são propriedades (ou atributos); Funções são métodos.
- Construtor: Bloco de código especial (constructor) chamado durante a criação do objeto, usado para inicializar atributos.
- `this`: Palavra reservada que se refere à instância atual da classe, usada para acessar propriedades e métodos.
- Propriedades de Parâmetro: Sintaxe simplificada para definir e inicializar propriedades diretamente no parâmetro do construtor, usando modificadores de visibilidade ou `readonly`.
- Modificadores de Visibilidade (Encapsulamento): Controlam o acesso aos membros da classe, sendo o principal recurso para o encapsulamento.

Técnicas de Programação I – Professor Henrique Louro

- public: Padrão. Acesso de qualquer lugar (classe, instância, subclasses). Representado por + na UML.
- private: Acesso restrito apenas dentro da própria classe onde foi definido. Não acessível por instâncias externas ou subclasses. Representado por - na UML.
- protected: Acesso permitido na classe e em suas subclasses que a herdam. Não acessível por instâncias externas. Representado por # na UML.
- readonly: Usado para declarar propriedades de somente leitura. O valor pode ser definido na declaração ou no construtor, mas não pode ser alterado depois.
- static: Define membros associados à própria classe, não às instâncias (objetos).
 - Acesso: Acessado diretamente pela classe (ex: Classe.membro).
 - Uso: Útil para métodos utilitários (classes utilitárias) ou dados compartilhados por todas as instâncias. Representado por sublinhado ou <<static>> na UML.
- Getters e Setters: Métodos especiais que controlam o acesso (leitura get) e a atribuição (escrita set) de propriedades, permitindo adicionar lógica personalizada (validações, formatações). São acessados como se fossem propriedades, sem parênteses.

IV. Herança e Polimorfismo

- Herança (Inheritance): Recurso da POO para reutilização de código. Permite que uma subclasse (classe filha) herde propriedades e métodos de uma superclasse (classe pai).
 - Sintaxe: Usa a palavra-chave extends.
 - Construtor da Subclasse: A primeira instrução deve ser a chamada ao construtor da superclasse, usando super().
 - Palavra super: Usada na subclasse para referenciar o objeto ou chamar métodos/construtores da superclasse.
 - Limitação: TS não suporta herança múltipla.
- Sobrescrita (Overriding): Permite que uma subclasse substitua a implementação de um método ou propriedade herdado da classe base por uma implementação própria. O método sobrescrito deve ter a mesma assinatura (nome e parâmetros).
- Sobrecarga (Overloading): Permite que uma classe tenha vários métodos (ou construtores) com o mesmo nome, mas com assinaturas diferentes (diferentes parâmetros ou tipos de retorno). A implementação real deve ser a última assinatura definida.
- Polimorfismo (Polymorphism): Significa "muitas formas". Permite que classes abstratas representem o comportamento de classes concretas. Uma mesma instrução (objeto.metodo()) pode ter execuções distintas dependendo do tipo do objeto. A sobrescrita é a principal técnica para alcançar o polimorfismo.

Técnicas de Programação I – Professor Henrique Louro

V. Abstração e Tipos de Contrato

- Classe Abstrata: Definida com `abstract`.
 - Restrições: Não pode ser instanciada diretamente com `new`.
 - Métodos Abstratos: Podem ter métodos sem corpo (sem implementação). Subclasses que estendem classes abstratas são obrigadas a implementar todos esses métodos abstratos.
 - UML: Nome em itálico e estereótipo `<<abstract>>`.
- Interfaces: Definidas com `interface`.
 - Contrato: Define uma estrutura mínima (contrato) de um objeto (propriedades e métodos sem corpo).
 - Implementação: Classes usam `implements` para seguir o contrato da interface.
 - Visibilidade: Não possuem modificadores de visibilidade; os membros implementados na classe são implicitamente públicos.
 - Uso em JSON: Usadas para definir a estrutura de objetos literais JSON.
 - UML: Nome em itálico e estereótipo `<<interface>>`. Relação de implementação é linha pontilhada com seta vazada.
- Type Alias (type): Similar a `*interface*`, usado para definir estruturas de tipos.
- Tipos Genéricos (Generics): Permitem criar componentes reutilizáveis (classes, funções, interfaces) que operam com diferentes tipos de dados (`<T>`), promovendo flexibilidade e segurança de tipo em tempo de compilação.

VI. Tratamento de Exceções

- Exceções: Erros que ocorrem durante a execução do programa.
- Bloco `try...catch...finally`:
 - `try`: Envolve o código que pode gerar exceção.
 - `catch`: Contém o código que será executado se uma exceção for capturada.
 - `finally`: Opcional. Contém código que sempre será executado, ocorrendo ou não a exceção.
 - `throw`: Instrução usada para lançar uma exceção em tempo de execução, interrompendo o fluxo normal. O objeto de exceção é geralmente criado usando `new Error()`.
 - Erros Personalizados: É possível criar tipos de erro customizados estendendo a classe base `Error`.